

**INDUSTRY INTERNSHIP
COGNIZANT
ON
ADOBE EXPERIENCE MANAGER (AEM)
DEVELOPMENT TRAINING**

*Internship Report submitted to
Shri Ramdeobaba College of Engineering & Management, Nagpur
in partial fulfillment of requirement for the award of degree of*

Bachelor of Technology

In
COMPUTER SCIENCE ENGINEERING

By
Raghav Bajaj

Under the guidance of

Prof. Leena Rokde



Department of Computer Science Engineering
Shri Ramdeobaba College of Engineering & Management, Nagpur 440 013
*(An Autonomous Institute affiliated to Rashtrasant Tukdoji Maharaj Nagpur University,
Nagpur)*

May 2026

**SHRI RAMDEOBABA COLLEGE OF ENGINEERING &
MANAGEMENT, NAGPUR**

*(An Autonomous Institute Affiliated to Rashtrasant Tukdoji Maharaj Nagpur University,
Nagpur)*

Department of Computer Science Engineering

CERTIFICATE

This is to certify that the Internship Report on "**Adobe Experience Manager (AEM) Development Training**" is a bonafide work of **Raghav Bajaj**, a student of Bachelor of Technology in Computer Science Engineering, submitted to Rashtrasant Tukdoji Maharaj Nagpur University, Nagpur in partial fulfillment of the award of the degree of Bachelor of Technology in Computer Science and Engineering. The internship has been carried out at Cognizant Technology Solutions, Siruseri Campus, Chennai during the academic year 2025–2026.

Date: _____

Place: Nagpur

Prof./Dr. _____

Industry Mentor

Cognizant Technology Solutions
Siruseri Campus, Chennai

Dr. P. Voditel

H.O.D

Department of Computer Science
Engineering

Prof./Dr. _____

Internal Guide

Department of Computer Science
Engineering

Dr. Parag Jawarkar

Dean-CDPC

Dr. M. B. Chandak

Principal

DECLARATION

I, Raghav Bajaj, hereby declare that the Internship Report titled "Adobe Experience Manager (AEM) Development Training" submitted herein, has been carried out at Cognizant Technology Solutions, Siruseri Campus, Chennai. The work presented in this report is original, carried out by me, and has not been submitted earlier as a whole or in part for the award of any degree, diploma, or certificate at this or any other institution or University.

The information and data presented in this report are based on my personal learning experience, training sessions, assessments, and hands-on practice conducted during the internship period from 31 January 2026 to 26 May 2026. Any reference to existing literature or third-party resources has been duly acknowledged.

I understand and acknowledge the importance of academic integrity and solemnly affirm that this report is an authentic account of my learning journey and does not violate any Non-Disclosure Agreement (NDA) signed with Cognizant Technology Solutions.

Date: _____

Place: Nagpur

(Signature)

Raghav Bajaj

B.Tech – Computer Science Engineering

Shri Ramdeobaba College of Engineering & Management, Nagpur

ACKNOWLEDGEMENT

I express my deepest gratitude to Cognizant Technology Solutions for providing me with the remarkable opportunity to undergo a structured industrial internship at their Siruseri Campus, Chennai. This experience has been instrumental in shaping my understanding of enterprise-grade software development and modern web content management technologies.

I am sincerely thankful to my Industry Mentor at Cognizant, whose patient guidance, continuous encouragement, and technical expertise greatly enriched my learning throughout the duration of the internship. The mentorship I received has been invaluable in bridging the gap between academic knowledge and industry-level practice.

I extend my heartfelt thanks to the training team at Cognizant for designing such a comprehensive and well-structured curriculum through the Tekstac platform, which provided a focused learning path covering core Java technologies, web technologies, and advanced enterprise frameworks such as Adobe Experience Manager (AEM). The hands-on problem statements and assessments on Tekstac greatly enhanced my practical understanding of the subjects covered.

I am deeply grateful to Dr. P. Voditel, Head of the Department, and Dr. Parag Jawarkar, Dean-CDPC, at Shri Ramdeobaba College of Engineering & Management, Nagpur, for their continued support and encouragement throughout the internship period. I also acknowledge the administrative and academic staff of the institution for their assistance in facilitating this industry internship program.

I would also like to place on record my appreciation for the Principal, Dr. M. B. Chandak, for his visionary leadership in ensuring that students receive practical industry exposure as part of their undergraduate curriculum.

Finally, I thank my family and peers for their constant moral support and motivation throughout this challenging yet rewarding internship journey. This experience

has not only expanded my technical skill set but has also instilled in me a professional work ethic and a deeper appreciation for collaborative software development.

Raghav Bajaj

ABSTRACT

This report documents the industrial internship undertaken by Raghav Bajaj, a student of Bachelor of Technology in Computer Science Engineering from Shri Ramdeobaba College of Engineering & Management, Nagpur, at Cognizant Technology Solutions, Siruseri Campus, Chennai, during the period of 31 January 2026 to 26 May 2026. The internship was organized as part of the Cognizant induction and training program for newly onboarded engineering graduates and was structured into two sequential stages of comprehensive technical training.

Stage 1 of the training encompassed a broad spectrum of foundational and intermediate web and enterprise technologies, including HTML5 and CSS3 for front-end development, JavaScript for dynamic client-side scripting, ANSI SQL for relational database management, Core Java for object-oriented programming, Java 8 for functional programming paradigms, JDBC for database connectivity, JSP and Servlets for server-side development, and auxiliary tools such as Java Logger, Log4J, JMS, Java Mail, and Maven for enterprise-grade application support.

Stage 2 focused on Adobe Experience Manager (AEM), a leading enterprise Content Management System (CMS) used extensively in digital experience platforms. Topics covered under Stage 2 included AEM architecture, Maven-based project setup, content authoring, component and template development, HTL (HTML Template Language, also known as Sightly), Sling Models, Client Libraries, Experience Fragments, Content Fragments, Touch UI and Dialogs, OSGi Services, Sling Servlets, user management, tagging, and AEM Assets. The learning was supported by hands-on assignments on the Tekstac platform and culminated in a formal multiple-choice assessment at the end of Stage 1.

This report elaborates on the training methodology, the depth of technology coverage, practical applications, and the overall outcomes in terms of professional and technical skill development. The internship has significantly enhanced the intern's capabilities in enterprise Java development and AEM-based content management, preparing them for real-world software engineering roles.

TABLE OF CONTENTS

S. No.	Content	Page No.
1.	Introduction to the organization	8
2.	Internship objectives	10
3.	Technologies and tools used	12
4.	Detailed learning description- Stage 1	16
5.	Detailed learning description- Stage 2(AEM)	21
6.	Training Methodology	28
7.	Role as an Intern	31
8.	Training and Project Flow	32
9.	Applications of Learned Technologies	34
10.	Outcomes and Skills Gained	36
11.	Conclusions	38

CHAPTER 1: INTRODUCTION TO THE ORGANIZATION

1.1 Overview of Cognizant Technology Solutions

Cognizant Technology Solutions Corporation is one of the world's foremost multinational technology companies, specializing in information technology services, consulting, and business process outsourcing. Founded in 1994 as a subsidiary of the American healthcare company Dun & Bradstreet, Cognizant was spun off as an independent entity and has since grown into a Fortune 200 company with a global workforce of over 300,000 professionals operating in more than 40 countries. Headquartered in Teaneck, New Jersey, USA, Cognizant is listed on the NASDAQ Global Select Market under the ticker symbol CTSH.

The company operates through a client-centric model, delivering technology-driven business transformation solutions across a wide array of industries including financial services, healthcare, manufacturing, retail, communications, and the public sector. Cognizant's service offerings span the full spectrum of digital transformation, encompassing cloud computing, artificial intelligence, data analytics, enterprise application development, quality engineering, and cybersecurity.

1.2 Cognizant's India Operations and Siruseri Campus, Chennai

India represents Cognizant's largest delivery hub and talent base, with major development centers located in Chennai, Pune, Hyderabad, Bengaluru, Kolkata, and Coimbatore, among others. The Chennai facility, particularly the Siruseri campus, stands as one of the flagship development centers of the company. Situated in the SIPCOT IT Park at Siruseri on the Old Mahabalipuram Road (OMR), this campus is a state-of-the-art facility housing tens of thousands of professionals working across various service lines and client engagements.

The Siruseri campus is equipped with modern infrastructure supporting enterprise software development, including advanced collaboration tools, dedicated training environments, high-speed connectivity, and ergonomic workspaces. The campus also houses Cognizant's internal learning and development infrastructure, including the

Cognizant Academy, which manages structured onboarding and continuous skilling programs for new hires and existing employees alike.

1.3 Cognizant's Technology and Innovation Philosophy

Cognizant prides itself on maintaining a culture of innovation and continuous learning. The company's digital transformation frameworks are designed to help enterprises navigate the complexities of the modern digital economy. In recent years, Cognizant has made significant investments in building competencies around cloud-native development, AEM-based digital experience platforms, and Java enterprise solutions.

As part of its commitment to talent development, Cognizant maintains a rigorous onboarding and training program for engineering graduates who join the company through campus recruitment. This program, delivered through the Cognizant Learning Management System and the Tekstac platform, ensures that new hires are equipped with the technical depth and breadth required to contribute effectively to client engagements from day one.

1.4 Cognizant's Internship and Training Framework

For B.Tech graduates joining Cognizant, the organization follows a structured induction and training schedule that spans several months. This program is organized into sequential training stages, each building upon the knowledge acquired in the previous stage. The training leverages a combination of instructor-led sessions, self-paced digital learning, hands-on lab exercises, and formal assessments to ensure comprehensive coverage of the required skill sets.

The training is delivered through the Tekstac platform, which provides curated learning content, interactive coding exercises, and assessment modules. This structured approach ensures that trainees develop not only theoretical understanding but also practical skills that are directly applicable to real-world project scenarios.

CHAPTER 2: INTERNSHIP OBJECTIVES

2.1 Primary Objectives

The internship at Cognizant Technology Solutions was undertaken with the following primary objectives:

- To acquire comprehensive knowledge of enterprise Java web development technologies, spanning front-end technologies (HTML5, CSS3, JavaScript), back-end server-side technologies (Core Java, Java 8, JDBC, JSP, Servlets), and database management (ANSI SQL), thereby developing the ability to design and implement complete, functional web applications.
- To gain in-depth expertise in Adobe Experience Manager (AEM), one of the industry's leading enterprise-grade content management systems, by understanding its architecture, development model, and deployment patterns used in large-scale digital experience platforms.
- To develop proficiency in practical application development by solving real-world problem statements on the Tekstac platform, ensuring that theoretical knowledge is translated into applied, implementable code.
- To successfully complete the Stage 1 formal assessment conducted by Cognizant, validating the learning outcomes of the first phase of training and demonstrating readiness for advanced technology modules.
- To understand the professional working environment, software development methodologies, and collaborative practices adopted in a global technology organization such as Cognizant.

2.2 Secondary Objectives

Beyond the primary technical learning objectives, the internship also aimed to accomplish the following secondary goals:

- To develop an understanding of enterprise software development practices, including the use of build tools (Maven), logging frameworks (Log4J, Java Logger), and messaging systems (JMS), which are commonly employed in large-scale production environments.
- To build an appreciation for the importance of clean, maintainable, and well-documented code through exposure to Cognizant's internal coding standards and best practices.

-
- To enhance problem-solving capabilities through exposure to structured programming assignments that simulate the complexity of real industry scenarios.
 - To develop soft skills including time management, technical communication, and self-directed learning, which are essential for success in a professional technology environment.

2.3 Expected Outcomes

At the conclusion of the internship, the following outcomes were expected to have been achieved: a strong foundation in enterprise Java development, practical experience in AEM component development and content management workflows, successful performance in formal competency assessments, and a readiness to contribute to live client projects within the Cognizant delivery model.

CHAPTER 3: TECHNOLOGIES AND TOOLS USED

3.1 Front-End Technologies

3.1.1 HTML5

HTML5 (HyperText Markup Language version 5) is the foundational markup language of the World Wide Web, used to structure and present content on web pages. During the internship, training covered both fundamental and advanced HTML5 features including semantic elements (such as header, footer, article, section, nav, and aside), forms with HTML5 validation attributes, multimedia integration using audio and video tags, canvas for 2D graphics, and the Web Storage API.

3.1.2 CSS3

Cascading Style Sheets version 3 (CSS3) was studied in depth, covering selectors (including pseudo-classes and pseudo-elements), the box model, positioning, flexbox layout, CSS Grid, transitions and animations, media queries for responsive design, and CSS variables. Responsive web design principles were practiced through building layouts that adapt seamlessly across various screen sizes and devices, an essential skill for modern enterprise UI development.

3.1.3 JavaScript

JavaScript training encompassed core language fundamentals (data types, control structures, functions, scope, and closures), Document Object Model (DOM) manipulation for dynamic web interaction, event handling, JSON parsing and serialization, object-oriented programming in JavaScript (prototypes, constructor functions, ES6 classes), and advanced array operations including map, filter, and reduce. These skills were critical for understanding the client-side scripting component of web application development.

3.2 Database Technology – ANSI SQL

Structured Query Language (SQL) training followed the ANSI standard and covered all key aspects of relational database management: Data Definition Language (DDL) operations for creating and modifying schemas, Data Manipulation Language (DML) for CRUD operations, complex JOIN operations (INNER, LEFT, RIGHT, FULL OUTER, SELF), subqueries, aggregate functions (COUNT, SUM, AVG, MIN, MAX),

GROUP BY and HAVING clauses, and database normalization up to Third Normal Form (3NF). The ability to write efficient SQL queries is a fundamental competency for any enterprise application developer.

3.3 Core Java and Java 8

3.3.1 Core Java

Core Java training provided a thorough grounding in the Java programming language, including object-oriented programming principles (encapsulation, inheritance, polymorphism, abstraction), interfaces and abstract classes, exception handling (checked vs. unchecked exceptions, custom exceptions, try-with-resources), the Java Collections Framework (ArrayList, LinkedList, HashMap, HashSet, TreeMap, LinkedHashMap, Iterator), I/O streams, and multithreading (Thread class, Runnable interface, synchronization, thread lifecycle, and the `java.util.concurrent` package including Executors, Future, and CountdownLatch).

3.3.2 Java 8

Java 8 training introduced the functional programming features added to the language with JDK 8. Key topics included Lambda Expressions, the Stream API for declarative processing of collections (filter, map, collect, reduce, flatMap), Optional class for null-safe programming, the new Date/Time API (LocalDate, LocalDateTime, ZonedDateTime), method references, and parallel streams for leveraging multi-core processor capabilities. These features represent a fundamental shift in Java programming style and are extensively used in modern enterprise Java development.

3.4 Server-Side Java Technologies

3.4.1 JDBC

Java Database Connectivity (JDBC) is the standard Java API for connecting to relational databases. Training covered the JDBC architecture, establishing database connections using DriverManager and DataSource, executing SQL queries using Statement, PreparedStatement, and CallableStatement, processing ResultSet objects, managing transactions, and implementing the DAO (Data Access Object) design pattern for clean separation of database logic from business logic.

3.4.2 JSP and Servlets

Java Server Pages (JSP) and Servlets are the fundamental building blocks of Java EE web applications. Servlet training covered the servlet lifecycle (init, service, destroy), handling HTTP GET and POST requests, request parameters and attributes, session management using HttpSession, cookies, and URL rewriting, and servlet filters and listeners. JSP training covered JSP directives, scriptlets, expressions, declarations, implicit objects, JSTL (JSP Standard Tag Library), and the MVC (Model-View-Controller) architecture for structuring Java web applications.

3.5 Enterprise Java Utilities

Additional enterprise Java utilities covered during Stage 1 included Java Logger and Log4J for application logging (configuring log levels, appenders, and layouts), Java Message Service (JMS) for asynchronous messaging between Java applications (queue-based and topic-based messaging models), Java Mail API for programmatic email generation and sending, and Apache Maven for project build management (dependency management, project lifecycle, POM configuration, and plugin usage). These tools are standard components of enterprise Java application stacks.

3.6 Adobe Experience Manager (AEM)

Adobe Experience Manager is an enterprise-grade Content Management System (CMS) and Digital Asset Management (DAM) platform built on the Apache Sling web framework, the OSGi component model (implemented via Apache Felix), and the Java Content Repository (JCR) standard (implemented via Apache Jackrabbit OAK). AEM enables organizations to create, manage, and deliver personalized digital experiences across web, mobile, and other channels. The technology stack employed in AEM development includes HTL (HTML Template Language), Sling Models, OSGi Services, Sling Servlets, and Maven-based project structures.

3.7 Development Tools and Platforms

Throughout the internship, various development tools were used: IntelliJ IDEA and Eclipse as Java IDEs, Apache Maven for build automation, Git for version control,

the Tekstac platform for learning content delivery and assessment, the AEM development environment (AEM Author and Publish instances), CRXDE Lite for JCR repository exploration, and the OSGi Web Console for bundle and configuration management.

CHAPTER 4: DETAILED LEARNING DESCRIPTION – STAGE 1

4.1 HTML5 and CSS3 – Front-End Foundation

The training in HTML5 and CSS3 began with an exploration of the evolution of web standards and the rationale behind HTML5's new semantic elements. Unlike earlier versions of HTML, which relied heavily on generic `div` and `span` tags for structuring content, HTML5 introduced purpose-specific elements that convey meaning both to browsers and to search engines. During the training, extensive practice was undertaken in building well-structured web pages using semantic elements, ensuring that the resulting markup was both human-readable and accessible.

CSS3 training moved beyond basic styling to cover advanced layout mechanisms. The Flexbox model was studied in detail, including the concepts of flex containers and flex items, the main axis and cross axis, alignment properties (`justify-content`, `align-items`, `align-self`), and flex sizing (`flex-grow`, `flex-shrink`, `flex-basis`). CSS Grid was subsequently introduced as a two-dimensional layout system, providing even greater control over complex page layouts. Responsive design was achieved through the strategic application of media queries, enabling the creation of layouts that adapt gracefully to different viewport sizes.

4.2 JavaScript – Client-Side Interactivity

The JavaScript module was one of the most extensive in Stage 1, reflecting the language's central role in modern web development. Training commenced with core JavaScript syntax and progressed through increasingly complex topics. DOM manipulation training involved selecting elements using `querySelector` and `querySelectorAll`, modifying element properties and styles programmatically, adding and removing DOM nodes, and handling user events using `addEventListener`. This provided a practical understanding of how JavaScript creates dynamic, interactive web pages.

The training on JavaScript objects covered object literal notation, constructor functions, the prototype chain, and ES6 class syntax. JSON (JavaScript Object Notation) was studied as both a data interchange format and as a means of representing structured

data within JavaScript applications. Array manipulation using the functional programming methods introduced in ES6 (map, filter, reduce, find, findIndex, every, some) was practiced extensively through problem statements on Tekstac, reinforcing the concept of writing concise, declarative code.

4.3 ANSI SQL – Database Management

Database training using ANSI SQL covered the complete lifecycle of data management within a relational database management system. The training began with understanding relational database concepts – tables, rows, columns, primary keys, foreign keys, and indexes – before moving into practical SQL query writing. CRUD operations (Create, Read, Update, Delete) were practiced thoroughly, forming the backbone of most data-driven application interactions.

Advanced SQL topics included multi-table JOIN operations, which are essential for querying normalized databases. Subqueries (both correlated and non-correlated) were studied for complex filtering and data retrieval scenarios. Aggregate functions were used in conjunction with GROUP BY and HAVING clauses for generating summary reports. Database normalization theory (1NF, 2NF, 3NF) was studied to understand how to design schemas that minimize redundancy and ensure data integrity. These SQL skills directly supported the JDBC training that followed.

4.4 Core Java – Object-Oriented Programming

Core Java training provided the deepest and most rigorous treatment of any module in Stage 1, given its foundational importance to all subsequent Java-based technologies. Object-Oriented Programming principles were not merely introduced theoretically but were applied consistently through progressively complex programming exercises. Encapsulation was practiced through the disciplined use of access modifiers and getter/setter methods. Inheritance was explored through class hierarchies and method overriding. Polymorphism was demonstrated through both compile-time (method overloading) and runtime (method overriding) scenarios. Abstraction was implemented through both abstract classes and interfaces.

Exception handling training covered the full exception hierarchy in Java, the difference between checked and unchecked exceptions, best practices for exception handling (avoiding empty catch blocks, using specific exception types), creating custom exception classes, and the try-with-resources construct introduced in Java 7 for automatic resource management. The Java Collections Framework was studied comprehensively, with particular attention paid to selecting the appropriate collection type for different use cases – ArrayList vs. LinkedList for sequential access, HashMap vs. TreeMap for key-value storage, and HashSet vs. LinkedHashSet for maintaining unique elements.

Multithreading training covered thread creation (extending Thread vs. implementing Runnable), thread lifecycle management, the synchronized keyword for mutual exclusion, the volatile keyword for visibility, inter-thread communication using wait/notify, and the high-level concurrency utilities in java.util.concurrent. Practical exercises included implementing producer-consumer patterns using blocking queues and managing thread pools using ExecutorService.

4.5 Java 8 – Functional Programming

The Java 8 module represented a paradigm shift in programming style, introducing functional programming concepts into the Java ecosystem. Lambda expressions were first introduced as a concise way to implement functional interfaces (interfaces with a single abstract method). The Functional Interface concept was explored through the built-in functional interfaces in java.util.function (Function, Predicate, Supplier, Consumer), demonstrating how lambdas enable the passing of behavior as method arguments.

The Stream API was the most extensively practiced feature of Java 8. Streams provide a declarative, high-level abstraction for processing sequences of data. Training covered stream creation (from collections, arrays, and Stream.of()), intermediate operations (filter, map, flatMap, distinct, sorted, peek, limit, skip), and terminal operations (collect, forEach, reduce, count, min, max, anyMatch, allMatch, noneMatch, findFirst, findAny). Collectors were studied in detail, including groupingBy, partitioningBy, joining, toMap, and toList. Parallel streams were explored to understand

how the Stream API can leverage multiple processor cores for improved performance on large datasets.

4.6 JDBC – Database Connectivity

JDBC training was conducted as a bridge between the database knowledge gained in the SQL module and Java for back-end development. The DAO pattern was implemented to encapsulate all database interactions, demonstrating how to write clean, testable, and maintainable data access code. Transaction management using `Connection.commit()` and `Connection.rollback()` was practiced ensuring data consistency in multi-step operations. Connection pooling concepts were introduced to explain how production systems efficiently manage database connections under concurrent load.

4.7 JSP and Servlets – Server-Side Web Development

JSP and Servlet training brought together the front-end and back-end knowledge developed thus far to create complete, functional web applications. A small web application was implemented as part of the Tekstac problem statements, applying the MVC pattern with Servlets acting as controllers, JSPs as views, and Java model classes encapsulating business data. Session management was implemented to track user state across multiple requests. The request-response lifecycle of a Java web application – from the initial browser request through the servlet container processing to the JSP rendering and response delivery – was traced in detail.

4.8 Enterprise Java Utilities

The enterprise utilities covered at the end of Stage 1 provided exposure to tools commonly encountered in the production of Java applications. Log4J was configured with both file and console appenders, different log levels, and rolling file policies to simulate production logging behavior. The Java Logger (`java.util.logging`) was compared with Log4J to understand the trade-offs between them. JMS was used to implement a simple message of producer and consumer, demonstrating the decoupling benefits of asynchronous messaging. Java Mail was used to programmatically construct and send email messages. Maven was used throughout all exercises to manage project

dependencies and build lifecycle, providing hands-on experience with a tool that is ubiquitous in enterprise Java development.

CHAPTER 5: DETAILED LEARNING DESCRIPTION – STAGE 2 (AEM)

5.1 Introduction to Adobe Experience Manager

Adobe Experience Manager (AEM) is an enterprise Content Management System built on a powerful open-source technology stack. At its core, AEM runs on Apache Sling, a web framework that maps HTTP requests content resources stored in a Java Content Repository (JCR). The JCR, implemented in AEM through Apache Jackrabbit OAK, provides a hierarchical, node-based storage model that stores all content – pages, components, templates, assets, and configurations – as structured nodes with properties.

The runtime environment for AEM is Apache Felix, an OSGi framework that provides a modular, dynamic service-oriented architecture. OSGi (Open Service Gateway initiative) allows AEM's functionality to be organized into discrete, independently deployable bundles (JAR files), each of which can be started, stopped, and updated without requiring a full application restart. This modularity is central to AEM's extensibility and is what enables developers to add custom functionality through custom OSGi services and components.

5.2 AEM Architecture

AEM operates in two primary modes – Author and Publish. The Author instance is used by content editors to create, edit, and manage digital content through a visual, browser-based interface known as the Touch UI (based on Coral UI and Granite UI). Once content is ready for public consumption, it is replicated from the Author instance to one or more Publish instances, which serve the content to end users. A Dispatcher (a web server module, typically Apache HTTP Server) sits in front of the Publish instances, providing caching and security capabilities.

Content in AEM is organized as a JCR node tree under `/content` for pages, `/content/dam` for digital assets, `/apps` for application code (components, templates, client libraries), `/conf` for configurations, and `/libs` for out-of-the-box AEM functionality. Developers follow the principle of overlaying `/libs` nodes under `/apps` rather than

modifying `/libs` directly, ensuring that custom configurations are preserved across AEM upgrades.

5.3 Maven Project Setup in AEM

AEM projects are typically initialized using the AEM Maven Archetype, which generates a multi-module Maven project structure. The standard project structure includes a core module (containing Java code – OSGi services, Sling Models, Sling Servlets), a `ui.apps` module (containing AEM-specific files such as component definitions, dialog XML, HTL scripts, and client libraries), a `ui.content` module (containing initial content and configurations), a `ui.config` module (for OSGi configurations), and a `ui.frontend` module (for modern front-end tooling integration). This modular structure enforces a clean separation of concerns and facilitates collaborative development.

Maven is used to build and deploy the AEM project. The Content Package Maven Plugin handles packaging of the `ui.apps` and `ui.content` modules into CRX packages, which are then installed into the AEM repository using the Package Manager or Maven deployment commands. During training, the complete Maven build and deployment cycle was practiced, reinforcing understanding of the project structure and deployment process.

5.4 AEM Authoring, Components, and Templates

Content Authoring in AEM is performed through a WYSIWYG (What-You-See-Is-What-You-Get) editor available in the Sites console. Pages are created based on Templates, which define the allowed components and the initial page structure. Editable Templates, introduced in AEM 6.x, allow template authors to define page structure and component policies through a visual template editor, providing greater flexibility compared to the older Static Templates.

AEM Components are the fundamental building blocks of AEM pages. Each component is a node in the JCR repository under `/apps`, associated with a dialog definition (for author configuration), an HTL script (for rendering), and optionally a Sling Model (for business logic). Training covered the anatomy of an AEM component in detail, including how component dialogs are defined using Granite UI form fields, how

component policies control which components are allowed in specific containers, and how inheritance works through the component hierarchy.

5.5 HTL (HTML Template Language / Sightly)

HTL, formerly known as Sightly, is the preferred server-side templating language for AEM component development. HTL is a secure templating language that runs server-side and automatically applies context-aware XSS escaping to all output, significantly reducing the risk of cross-site scripting vulnerabilities. The language syntax consists of HTML-compliant attributes (data-sly-* attributes) that control conditional rendering, list iteration, resource inclusion, and template composition.

Key HTL constructs studied included data-sly-use (for instantiating Sling Models or Use-API objects), data-sly-list (for iteration), data-sly-test (for conditional rendering), data-sly-include (for including other scripts), data-sly-resource (for including resources from the JCR), and data-sly-call (for invoking HTL templates). Java-based Sling Models are the preferred approach for complex logic, keeping HTL templates clean and focused purely on presentation.

5.6 Sling Models

Sling Models are Plain Old Java Objects (POJOs) annotated with Sling-specific annotations that allow them to be adapted from Sling Request or Resource objects. They serve as the business logic layer in AEM component development, decoupling Java logic from HTL templates. A Sling Model is annotated with `@Model`, specifying the adaptable type (`SlingHttpServletRequest` or `Resource`) and the interface it implements. Fields within the model are injected using annotations such as `@ValueMapValue` (for JCR property injection), `@ChildResource`, `@RequestAttribute`, `@OSGiService` (for injecting OSGi services), and `@SlingObject`.

During training, multiple custom Sling Models were developed to back HTL components, demonstrating how to retrieve content from the JCR, perform business logic, and expose data to the template layer through getters. The `@PostConstruct` annotation

was used to define initialization logic that runs after all injections are complete, enabling validation and data transformation.

5.7 Custom Component Development

The practical development of custom AEM components was one of the most significant hands-on activities in Stage 2. Custom components were created from scratch, starting with the JCR node definition in the `/apps` namespace. Each component node contains properties specifying the component group, title, and associated resource type. The dialog was defined using the Granite UI dialog structure (`cq:dialog` node), which rendered in the Touch UI as an edit dialog for content authors.

A complete custom component development cycle involves: defining the component node, creating the `cq:dialog` with appropriate Granite UI field types (textfield, textarea, checkbox, pathfield, multifield), writing the HTL script for rendering, creating the backing Sling Model for logic, and applying CSS/JS via Client Libraries. Each of these steps was practiced through multiple component development exercises, building progressively more complex components.

5.8 Client Libraries

Client Libraries (ClientLibs) are AEM's mechanism for managing and serving CSS and JavaScript assets. They are defined as JCR nodes with the type `cq:ClientLibraryFolder` and contain CSS and JavaScript files, along with metadata files (`css.txt` and `js.txt`) that specify the order in which the files should be bundled. Client Libraries support categorization (through the `categories` property), dependency management (through the `dependencies` property), and embedding of multiple client libraries into a single library.

Training covered the inclusion of client libraries in HTL templates and the AEM Core Components' `HtmlPageItem` mechanism. The concept of author vs. publish client libraries was also explored, demonstrating how to include libraries only in the AEM authoring environment (for editor-specific styling) or only in the publish environment (for production styling).

5.9 Experience Fragments and Content Fragments

Experience Fragments (XFs) are reusable for page fragments that can contain both content and layout, designed to be embedded within AEM pages or exported to external channels. They allow content authors to create a piece of content once and reuse it across multiple pages or channels, ensuring consistency and reducing content duplication. Training covered the creation of Experience Fragment templates, the authoring of Experience Fragment variations, and the embedding of Experience Fragments into page components.

Content Fragments (CFs) are channel-agnostic, structured content objects that separate content from presentation. Unlike Experience Fragments, which contain layout information, Content Fragments consist purely of structured data defined by a Content Fragment Model. They are ideal for headless CMS use cases, where content needs to be consumed via APIs by various front-end clients. Training covered the creation of Content Fragment Models, authoring Content Fragment instances, and retrieving them via Sling Servlets.

5.10 Touch UI, Dialogs, and Validators

The Touch UI is AEM's modern, responsive authoring interface built on Coral UI (Adobe's design system) and Granite UI (a server-side rendering framework for Coral UI components). Component dialogs, Page Properties dialogs, and template editors are all rendered using Touch UI. Training covered the Granite UI form field types in depth, including how to configure field validation using Granite UI validator scripts, how to create multifield components for repeatable data entries, and how to use the Pathfield for JCR node selection.

Custom dialog validators were implemented using JavaScript and registered as Granite UI client libraries, demonstrating how to enforce content entry rules at the authoring layer before content is persisted to the JCR.

5.11 OSGi Services and Configuration

OSGi Services are the backbone of AEM's extensibility model. A custom OSGi Service is a Java interface implemented by one or more service implementation classes, annotated with `@Component` and `@Service` from the OSGi DS (Declarative Services) specification. Services are registered in the OSGi service registry and can be referenced (injected) by other components using `@Reference`.

OSGi Configuration allows service behavior to be configured externally through the OSGi Web Console or through repository-based OSGi configuration files (.cfg.json). A configurable OSGi service uses the `@Designate` annotation to associate the service with a configuration interface. Training involves creating configurable OSGi services for common utilities such as content retrieval, external API calls, and environment-specific configuration management.

5.12 Sling Servlets

Sling Servlets are Java servlets registered with the OSGi container and mapped to specific JCR resource paths or resource types using OSGi service properties. Unlike traditional Java Servlets mapped to URL paths, Sling Servlets follow Sling's resource-centric request resolution algorithm, making them more aligned with AEM's content-first model. Two registration approaches were studied: resource type-based registration (using `resourceTypes` and selectors) and path-based registration (using paths).

Training exercises involve creating Sling Servlets to expose JCR data as JSON APIs, implementing custom search endpoints using the JCR Query API (`QueryBuilder`), and processing POST requests for form submissions. The use of `SlingHttpServletRequest` and `SlingHttpServletResponse` objects was explored in detail.

5.13 Users, Permissions, and Tagging Framework

AEM's user and group management model is based on the Jackrabbit Access Control model, where each JCR node can have Access Control Entries (ACEs) that grant or deny specific privileges (read, write, replicate, etc.) to specific users or groups. Training covered the creation of service users (non-interactive users used by OSGi services for JCR access), the definition of service user mappings in OSGi configurations,

and the assignment of minimal necessary permissions following the principle of least privilege.

The AEM Tagging Framework provides a hierarchical taxonomy for categorizing content. Tags are stored as JCR nodes under `/content/cq:tags` and can be applied to any JCR resource. Training covered the creation and management of tag namespaces and individual tags, the application of tags to pages and assets, and the retrieval of tagged content using the Tag Manager API and JCR queries.

5.14 AEM Assets (Digital Asset Management)

AEM Assets is the Digital Asset Management (DAM) component of AEM, providing a centralized repository for storing, organizing, and managing digital assets such as images, videos, documents, and other binary files. Assets are stored under `/content/dam` in the JCR, with metadata stored as JCR properties on the asset node. Training covered the ingestion of assets into the DAM through the Assets UI, the configuration of asset processing workflows (for generating renditions, extracting metadata, and applying watermarks), and the retrieval of assets programmatically using the AssetManager API.

Asset metadata schemas were explored, demonstrating how to configure custom metadata fields for specific asset types. The Assets API was used within Sling Models and Sling Servlets to retrieve asset renditions and metadata, enabling the dynamic embedding of DAM assets within page components. This concluded the AEM curriculum covered during the internship period.

CHAPTER 6: TRAINING METHODOLOGY

6.1 Structured Curriculum Design

Cognizant's training program followed a meticulously designed, competency-based curriculum structure. The curriculum was divided into two major stages, each with clearly defined learning objectives, topic sequences, and assessment milestones. This sequenced approach ensured that each new topic built upon the foundation established by the preceding topics, creating a coherent and progressive learning experience. For instance, SQL knowledge was prerequisite to JDBC training, Core Java was prerequisite to Java 8, and all Stage 1 topics collectively formed the foundation for AEM development in Stage 2.

6.2 Tekstac Platform – Digital Learning and Assessment

The Tekstac platform served as the primary digital learning and assessment environment throughout the internship. Tekstac is a specialized Learning Management System (LMS) designed for technical training, providing curated learning modules, video content, and interactive coding exercises. Each topic in the curriculum was associated with a set of problem statements on Tekstac that required the application of the concepts covered. These problem statements ranged from straightforward implementation exercises to more complex scenarios that required combining knowledge from multiple topics.

The Tekstac platform also hosted the Stage 1 assessment, which was a Multiple Choice Question (MCQ) based examination covering all topics from Stage 1. The assessment was designed to test both conceptual understanding and practical application knowledge. Performance on the Tekstac platform was tracked and evaluated as part of the overall internship progress assessment.

6.3 Hands-On Practice and Problem Solving

A significant proportion of the training time was dedicated to hands-on practice. For each technology module, practical coding exercises were completed on the Tekstac platform, involving the implementation of functional programs and applications. This hands-on approach was invaluable in consolidating the theoretical knowledge covered in

the learning modules. Writing actual code – encountering compilation errors, debugging logic errors, and refactoring clarity – provided a depth of understanding that passive reading of documentation alone cannot be achieved.

For the AEM modules, hands-on practice involved setting up a local AEM development environment, creating Maven projects using the AEM Archetype, developing custom components end-to-end (from component node definition to HTL template and Sling Model), and deploying the projects to the local AEM Author instance for testing in the Touch UI.

6.4 Stage 1 Assessment

The Stage 1 Assessment was conducted on 19 March 2026 as a formal competency evaluation. The assessment was MCQ-based, covering all major topics from Stage 1 including HTML5, CSS3, JavaScript, ANSI SQL, Core Java, Java 8, JDBC, JSP, Servlets, and the enterprise utilities. The examination was conducted under supervised conditions within the Tekstac platform, with a defined time limit. Successful completion of the Stage 1 Assessment was a prerequisite for progression to Stage 2 training. The assessment was passed successfully on the first attempt, demonstrating adequate mastery of all Stage 1 competencies.

6.5 Interim Evaluation (AEM)

An Interim Evaluation was conducted on 20 April 2026, midway through the Stage 2 AEM training. Unlike the Stage 1 Assessment, which was a written MCQ examination on the Tekstac platform, the Interim Evaluation was structured as an online interview conducted by a Cognizant technical assessor. The evaluation tested the trainee's conceptual understanding and practical grasp of the AEM topics covered up to that point, including AEM architecture, component development, HTL, Sling Models, Client Libraries, OSGi Services, and Experience Fragments.

The grading system used for the Interim Evaluation was a three-tier colour-coded scheme: Green (highest performance), Amber (satisfactory, with areas for improvement), and Red (below expectation). This colour-coded model provides a quick, clear signal of

a trainee's readiness to progress and highlights any competency gaps that require targeted attention before the trainee is assigned to a live client project. A Green rating indicated strong conceptual clarity, accurate articulation of AEM concepts, and confident handling of interview questions. An Amber rating would suggest adequate but inconsistent knowledge, while a Red rating would flag a need for remedial review before progression.

The Interim Evaluation was completed with a green rating, reflecting a strong command of the AEM topics covered during Stage 2 training. This outcome validated the effectiveness of the hands-on learning approach and confirmed that the knowledge gained through daily practice, CRXDE Lite exploration, and Maven-based component development had translated into clear, communicable understanding. The Green rating also served as a positive indicator of readiness to proceed to the remaining AEM modules and, subsequently, to real project work within Cognizant's delivery organization.

6.6 Peer Collaboration and Knowledge Sharing

Although the training program was individually paced, there were opportunities for peer interaction and collaborative problem solving among the trainee's cohort. Discussions with fellow trainees on challenging problem statements, sharing of debugging approaches, and collaborative review of difficult AEM concepts contributed to a richer learning experience. This collaborative element also mirrored the team-based nature of real-world software development, where knowledge sharing and pair programming are common practices.

CHAPTER 7: ROLE AS AN INTERN

7.1 Position and Responsibilities

The internship at Cognizant was undertaken in the role of a Programmer Analyst Trainee – the designation assigned to engineering graduates who join Cognizant through campus recruitment and are undergoing the induction training program. In this capacity, the primary responsibility was to actively participate in the training program, demonstrate satisfactory progress on the Tekstac platform, complete all assigned problem statements, and achieve the required performance benchmarks in formal assessments.

Beyond the formal training obligations, the role also involved maintaining a professional work attitude, adhering to Cognizant's code of conduct and workplace policies, meeting daily attendance requirements at the Siruseri training facility, and engaging proactively with mentors and training personnel to resolve learning challenges.

7.2 Interaction with Mentors and Trainers

Regular interaction with industry mentors and Cognizant's training personnel was an integral part of the internship experience. Doubt-clearing sessions, periodic progress reviews, and one-on-one discussions with mentors provided guidance on both technical challenges and professional development. Feedback received from mentors was incorporated into subsequent learning activities, creating a continuous improvement cycle. The mentors also provided valuable industry perspective on how the technologies being studied are applied in real client engagements, bridging the gap between training content and real-world application.

CHAPTER 8: TRAINING AND PROJECT FLOW

8.1 Overall Training Flow

The internship training followed a sequential, phase-based flow organized across the full duration from 31 January 2026 to 26 May 2026.

Phase 1 – Onboarding and Orientation: Upon joining on 31 January 2026, the first week was dedicated to the induction process. This involved orientation sessions covering Cognizant's organizational structure, values, work policies, HR processes, and an introduction to the training program and its objectives. Access to the Tekstac platform, training environments, and internal systems was provisioned during this phase.

Phase 2 – Stage 1 Core Technology Training (February – March 2026): The bulk of February and the first half of March were devoted to Stage 1 training. The curriculum progressed in logical sequence: HTML5 and CSS3 → JavaScript → ANSI SQL → Core Java → Java 8 → JDBC → JSP and Servlets → Enterprise Utilities (Logging, JMS, Java Mail, Maven). Each topic of transition was marked by the completion of associated problem statements on Tekstac.

Phase 3 – Stage 1 Assessment (19 March 2026): A formal MCQ-based assessment was conducted on 19 March 2026, evaluating competency across all Stage 1 topics. Successful completion of this assessment validated the learning from Phase 2 and authorized progression to Stage 2.

Phase 4 – Stage 2 AEM Training (Late March – May 2026): Following the Stage 1 assessment, Stage 2 training commenced with Adobe Experience Manager. The progression within AEM training followed the natural dependency order: AEM Introduction and Architecture → Maven Project Setup → Authoring and Page Structure → Components and Templates → HTL → Sling Models → Custom Component Development → Client Libraries → Experience Fragments and Content Fragments → Touch UI and Dialogs → OSGi Services → Sling Servlets → Users and Permissions → Tagging Framework → AEM Assets.

Phase 5 – Interim Evaluation (20 April 2026): An online interview-based Interim Evaluation was held on 20 April 2026 to assess progress midway through Stage 2. The evaluation covered the AEM topics completed up to that point and used a Green-Amber-Red grading scheme, with Green representing the highest level of performance. The evaluation was completed with a green rating, confirming strong conceptual clarity and practical readiness to continue with the remaining AEM modules.

8.2 AEM Component Development Workflow

The development workflow for a custom AEM component, practiced repeatedly during Stage 2, follows a well-defined sequence. First, the component node structure is defined in `/apps`, including the component descriptor properties (`jcr:title`, `componentGroup`, `slings:resourceType`). Second, the `cq:dialog` node is created with Granite UI field definitions to power the author interface. Third, the HTL rendering script (`.html` file) is written to produce the component's HTML output. Fourth, a Sling Model Java class annotated with `@Model` is created to handle business logic and expose data to HTL. Fifth, a Client Library node is created with associated CSS and JavaScript for component styling and interactive behavior. Sixth, the project is built and deployed using Maven. Finally, the component is tested in the AEM Touch UI by adding it to a page via the Edit mode and verifying the dialog fields and rendered output.

8.3 Assessment and Progression Framework

Cognizant's training program employs a gated progression model, where advancement to each subsequent training stage requires demonstrated competency in the current stage. The Stage 1 Assessment served as the formal gate between Stage 1 and Stage 2. An additional Interim Evaluation, conducted as an online interview on 20 April 2026, served as a mid-Stage 2 checkpoint, using a Green-Amber-Red grading scheme to assess the trainee's understanding of AEM topics covered up to that point. Continuous performance monitoring on the Tekstac platform (through problem statement completion rates and scores) also contributed to the overall progress of evaluation. This multi-layered assessment framework ensures that trainees have the necessary foundational knowledge before being exposed to more advanced topics, reducing the risk of knowledge gaps that could impact performance on client projects.

CHAPTER 9: APPLICATIONS OF LEARNED TECHNOLOGIES

9.1 Enterprise Web Application Development

The enterprise Java skills acquired during Stage 1 directly support the development of web applications. The combination of HTML5, CSS3, and JavaScript for the front-end, Java Servlets and JSP for the back-end, JDBC for database interaction, and SQL for data storage represents a complete, functional technology stack for building Java EE web applications. Such applications are widely deployed in enterprise environments for internal tools, portals, and customer-facing services.

The MVC architectural pattern practiced through JSP and Servlet training is directly applicable to enterprise Java web development frameworks. The foundational understanding of the MVC pattern, request processing lifecycle, and session management provides the conceptual grounding necessary to quickly adapt to higher-level frameworks used in industry projects.

9.2 Digital Experience Platform Development

AEM skills are directly applicable to building and maintaining enterprise Digital Experience Platforms (DXPs). Many Fortune 500 companies use AEM as their primary web content management system, and Cognizant serves numerous such clients across industries including financial services, retail, healthcare, and manufacturing. Developers with AEM expertise can contribute to projects involving the development of custom AEM components, template configurations, content migration workflows, and AEM-as-a-Cloud-Service deployments.

The experience with Experience Fragments and Content Fragments is particularly relevant to modern headless CMS architectures, where AEM serves as the content hub, and various front-end frameworks consume the content through APIs. This pattern is increasingly adopted by organizations seeking omnichannel content delivery.

9.3 Service-Oriented and API Development

The OSGi Services and Sling Servlets knowledge gained during AEM training is conceptually transferable to service-oriented architecture (SOA) patterns. Understanding how to develop loosely coupled, independently deployable services is a foundational principle applicable across diverse back-end development contexts. Similarly, the experience with RESTful API design principles (demonstrated through Sling Servlet development) is applicable across the entire spectrum of back-end API development, regardless of the specific framework or platform used.

9.4 Data Processing and Functional Programming

The Java 8 Stream API and parallel programming knowledge acquired during Stage 1 has direct applications in data processing pipelines. The functional programming style enabled by streams, lambdas, and collectors is increasingly used in data processing contexts, providing a foundation for working with large-scale data transformation and analysis tasks. The Optional class and null-safe programming patterns learned in Java 8 directly contribute to writing more robust, production-grade code with fewer runtime errors.

9.5 Content Management and Digital Marketing

AEM's authoring capabilities, Experience Fragments, and Content Fragments training provides a direct understanding of enterprise content management workflows. This knowledge is applicable to roles involving the implementation, customization, and administration of AEM-based digital marketing platforms, including campaign management, personalization, and A/B testing scenarios enabled by AEM's integration with Adobe Target and Adobe Analytics. The Tagging Framework and AEM Assets knowledge supports digital asset management operations critical to large-scale marketing operations.

CHAPTER 10: OUTCOMES AND SKILLS GAINED

10.1 Technical Skills Acquired

The internship resulted in the acquisition of a comprehensive and directly industry-applicable technical skill set. In the domain of front-end development, proficiency was gained in HTML5 semantic markup, advanced CSS3 layout techniques (Flexbox and Grid), responsive design, and JavaScript DOM manipulation and functional array processing. These skills collectively enable the development of modern, standards-compliant, and responsive web user interfaces.

In the database domain, the ability to write complex, optimized SQL queries including multi-table joins, subqueries, and aggregate operations was developed. This skill is foundational to all data-driven application development, regardless of the specific database technology or ORM framework used.

In back-end Java development, substantial proficiency was gained in both traditional Java EE (Core Java, JDBC, JSP, Servlets) and modern Java 8 functional programming (Streams, Lambdas, Optional). These skills cover the full spectrum of enterprise Java development and are directly applicable to production-grade application development.

In AEM development, the internship resulted in practical competency across the complete AEM component development lifecycle, from project setup through Maven, to component authoring, HTL templating, Sling Model development, OSGi service development, Sling Servlet implementation, and content management through Experience Fragments, Content Fragments, and AEM Assets. This represents a highly specialized and commercially valuable skill set.

10.2 Professional Skills Developed

Beyond technical skills, the internship significantly contributed to professional development. Time management and self-discipline were strengthened through the structured daily training schedule (8:30 AM to 6:30 PM) and the need to maintain consistent progress on the Tekstac platform. Problem-solving and debugging skills were

honed through the process of resolving technical challenges encountered during hands-on exercises. Technical communication skills were developed through interactions with mentors and participation in group discussions.

The experience of working within the policies and cultural norms of a large multinational organization, including adherence to workplace standards and collaborative working practices, provided valuable insights into the professional environment of the technology industry.

10.3 Assessment Performance

Two formal evaluations stand as concrete, measurable outcomes of the internship. The Stage 1 Assessment, passed successfully on the first attempt on 19 March 2026, formally validated competency across all Stage 1 technologies. The Interim Evaluation conducted on 20 April 2026 — an online interview assessing AEM knowledge midway through Stage 2 — was completed with a Green rating, the highest grade in Cognizant’s Green-Amber-Red grading scheme. Together, these results confirm that the level of learning achieved across both stages meets the professional standards set by one of the world’s leading technologies companies and provide industry-recognized evidence of technical competency.

10.4 Career Readiness

Perhaps the most significant outcome of the internship is an enhanced state of career readiness. Prior to the internship, knowledge of technologies such as AEM, OSGi, Sling, and Java 8 was largely theoretical, gained through academic coursework. The internship has transformed this theoretical knowledge into practical, hands-on competency, directly applicable to real-world project work. The confidence gained through solving complex, real-world-style problem statements on Tekstac and implementing functional AEM components from scratch has substantially increased readiness to contribute to live client projects.

CHAPTER 11: CONCLUSION

The internship at Cognizant Technology Solutions, Siruseri Campus, Chennai, spanning from 31 January 2026 to 26 May 2026, has been an extraordinarily enriching and professionally transformative experience. The structured, two-stage training program has provided a comprehensive technical foundation that spans the complete spectrum of enterprise web application development, from front-end presentation technologies through server-side Java development to one of the world's leading enterprise Content Management Systems.

Stage 1 of the training successfully established a strong foundation in the core technologies of enterprise Java development. The progressive curriculum, moving from web fundamentals through to advanced Java 8 functional programming and enterprise utilities, ensured that each layer of knowledge reinforced and extended what was built before. The Stage 1 Assessment, passed successfully on 19 March 2026, formally validated this foundational competency.

Stage 2's focus on Adobe Experience Manager opened an entirely new dimension of technical knowledge – the world of enterprise Content Management Systems and Digital Experience Platforms. The depth of the AEM curriculum, covering everything from the JCR repository model and OSGi component framework through to custom component development, Sling Models, OSGi Services, and AEM Assets, has provided a level of AEM expertise that is directly applicable to real client delivery work. The Interim Evaluation conducted on 20 April 2026, an online interview assessed using Cognizant's Green-Amber-Red grading scheme, was completed with a Green rating – confirming that the AEM knowledge acquired through hands-on practice had been thoroughly understood and could be confidently articulated.

The hands-on methodology employed by Cognizant – combining structured digital learning through Tekstac with practical implementation exercises conducted across a rigorous 8:30 AM to 6:30 PM daily training schedule – proved highly effective in cementing technical understanding. The depth of engagement required to complete problem statements, implement functional components, and prepare formal assessments translated directly into practical, deployable competency.

On reflection, the internship has not only expanded the technical skill set significantly but has also instilled a professional mindset, a disciplined approach to self-directed learning, and a deeper appreciation for the complexity and sophistication of enterprise-grade software systems. The experience has substantially enhanced readiness to contribute meaningfully to Cognizant's client delivery work and serves as a solid launching pad for a productive career in software engineering.

In conclusion, this internship represents a pivotal milestone in the transition from academic learning to professional practice, and the knowledge, skills, and perspectives gained during this period will continue to guide and inform professional growth for years to come.

REFERENCES

- Adobe Systems Incorporated. (2024). Adobe Experience Manager Documentation. Adobe Experience League. Retrieved from <https://experienceleague.adobe.com/docs/experience-manager.html>
- Oracle Corporation. (2024). Java SE 8 API Documentation. Oracle Technology Network. Retrieved from <https://docs.oracle.com/javase/8/docs/api/>
- Apache Software Foundation. (2024). Apache Sling Documentation. Retrieved from <https://sling.apache.org/documentation.html>
- Apache Software Foundation. (2024). Apache Felix OSGi Framework Documentation. Retrieved from <https://felix.apache.org/documentation.html>
- Apache Software Foundation. (2024). Apache Maven Project Documentation. Retrieved from <https://maven.apache.org/guides/>
- Mozilla Developer Network. (2024). HTML5 Web API Reference. Retrieved from <https://developer.mozilla.org/en-US/docs/Web/HTML>
- Mozilla Developer Network. (2024). CSS3 Reference. Retrieved from <https://developer.mozilla.org/en-US/docs/Web/CSS>
- Mozilla Developer Network. (2024). JavaScript Reference. Retrieved from <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
- Bloch, J. (2018). Effective Java (3rd Edition). Addison-Wesley Professional.
- W3Schools. (2024). SQL Tutorial. Retrieved from <https://www.w3schools.com/sql/>
- Cognizant Technology Solutions. (2026). Cognizant Learning Platform – Tekstac Training Modules (Internal Resource).
- Oracle Corporation. (2024). Java EE 8 Tutorial – Servlets and JSP. Retrieved from <https://javaee.github.io/tutorial/>
- OSGi Alliance. (2024). OSGi Compendium Specification. Retrieved from <https://www.osgi.org/resources/specifications/>
- Adobe Inc. (2024). AEM WCM Core Components Documentation. Retrieved from <https://github.com/adobe/aem-core-wcm-components>
- Shalloway, A., & Trott, J. R. (2004). Design Patterns Explained: A New Perspective on Object-Oriented Design (2nd Edition). Addison-Wesley Professional.

